

Quantum Encoding Agents: A Natural Language Interface for Data Embedding Strategy Selection in Quantum Machine Learning

Ana Paula Appel
aappel@redhat.com
Red Hat, Inc.
USA

Abstract

Selecting an appropriate data encoding strategy is one of the most consequential and least-tooled decisions in Quantum Machine Learning (QML) pipelines. The choice of quantum feature map determines the structure of the Hilbert space into which classical data is embedded, directly shaping the expressibility of quantum kernels, the trainability of variational circuits, and the feasibility of execution on near-term hardware. Despite its central role, encoding selection is rarely addressed systematically: practitioners typically default to a single strategy—most often angle encoding—without analyzing how the structural characteristics of their data interact with the properties of each encoding family. This paper presents **Quantum Encoding Agents**, an open-source system that reformulates encoding selection as a natural language interaction. Given a dataset and an optional description of the QML task, the system analyzes the data profile, applies a hardware-aware recommendation policy grounded in the critical gate error threshold $p^* \approx 10^{-3}$, generates a complete copyable Qiskit circuit, produces a natural language justification in the user’s language (Portuguese or English), and computes the quantum kernel matrix with Kernel-Target Alignment (KTA) scoring—a label-aware scalar metric that measures how well the quantum kernel groups same-class samples and separates different-class samples in Hilbert space. When a numerical matrix is available, the system first estimates the correlation fractal dimension D_2 and selects original columns via FD-ASE [3, 10], using $q^* = \max(2, \lceil D_2 \rceil)$ as an *a priori* qubit budget so that angle and IQP maps are not recommended at a width where the fidelity kernel has already collapsed. The system is implemented as a FastAPI microservice, deployed on Red Hat OpenShift with NVIDIA OpenShift sandbox isolation, and exposed through OpenClaw agents with distinct epistemic personalities calibrated to different levels of user expertise. Evaluation confirms that the system correctly selects among seven encoding families—amplitude, angle, dense angle, IQP, basis, data re-uploading, and custom feature map—under realistic NISQ hardware constraints, and that KTA scores discriminate between encodings on benchmark datasets.

CCS Concepts

• **Computing methodologies** → **Quantum computing**; *Supervised learning*; *Natural language processing*; • **Security and privacy** → Access control.

Keywords

Quantum Machine Learning, Quantum Feature Maps, Data Encoding, Intrinsic Dimension, Fractal Dimension, AI Agents, NISQ Hardware, OpenShift, Natural Language Interface

1 Introduction

The practical adoption of Quantum Machine Learning faces a fundamental bottleneck that precedes model training: the problem of quantum data encoding. Before any quantum classifier, kernel method, or variational circuit can operate, classical data must be mapped into quantum states. This mapping—the encoding or feature map—is not a neutral preprocessing step. It defines the geometry of the state space, the structure of correlations captured between features, and the depth and qubit count of the resulting circuit. Different encodings impose radically different tradeoffs: amplitude encoding compresses n features into $\lceil \log_2 n \rceil$ qubits but requires a state preparation circuit of exponential depth; angle encoding produces shallow circuits with one qubit per feature but limited expressibility; IQP encoding [15] captures cross-feature correlations through diagonal unitaries and has strong theoretical guarantees for quantum kernels, but at greater circuit depth.

Despite the critical nature of this choice, the QML literature offers little guidance on encoding selection as a function of data characteristics. Practitioners encounter a combinatorial decision problem: seven or more encoding families, each with different qubit efficiency, circuit depth, hardware sensitivity, and affinity for different QML algorithms, must be matched against a dataset whose relevant properties—dimensionality, value type, distribution, sign—interact with the encoding in non-obvious ways. The seminal survey by Sammartino [28], reviewing 66 papers from 2017 to 2026, identifies this as the most underaddressed problem in applied QML, formalizes six encoding families, and derives a critical gate error rate threshold $p^* \approx 10^{-3}$ above which deep encodings become impractical on NISQ hardware.

This paper addresses the encoding selection problem through a different lens: rather than proposing a new encoding or a new theoretical framework, I build a practical agent system that makes existing knowledge actionable. I make the following contributions:

- (1) **A hardware-aware encoding recommendation engine** that applies the p^* threshold from [28], accounting for gate error rate, circuit depth budget, qubit count, and chip connectivity topology.
- (2) **A fractal qubit budget** [3] that estimates D_2 (LiBOC) and selects original columns with FD-ASE [10] *before* the recommendation tree and the labelled KTA search, so angle/IQP

are not proposed at a width the kernel cannot bear. The crop is the default; `apply_fractal_budget=false` still reports D_2 and the suggested columns while encoding the full table.

- (3) **Seven implemented encoding strategies** in Qiskit, including dense-angle encoding [28] and IQP encoding [15], which were identified as underrepresented in applied tooling.
- (4) **A natural language explanation layer** that generates structured justifications in Portuguese and English, citing concrete metrics (number of qubits, circuit depth, Kernel-Target Alignment (KTA), D_2 , q^*) derived from actual simulation results.
- (5) **A quantum kernel evaluation endpoint** that computes $K_{ij} = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$ for a dataset, returns KTA scores, ranks encodings when labels are available, and reports the operational kernel-alive diagnostic of [3] (near/far fidelities and mean off-diagonal) on the same matrix. `/v1/compare` also sweeps qubit width q under FD-ASE, PCA, and CSV-prefix views so q^* can be compared with the PCA-95% allocation.
- (6) **A multi-agent deployment architecture** on Red Hat OpenShift with NVIDIA OpenShell security sandboxes, exposing three OpenClaw agents with distinct communicative personalities for expert and novice audiences.
- (7) **A Kubeflow Pipeline** for Red Hat OpenShift AI that orchestrates the full workflow—data analysis, encoding recommendation, comparison, kernel evaluation, and MLflow artifact logging—as a reproducible five-stage DAG.

The system is released as open source.¹

2 Related Work

2.1 Quantum Data Encoding Strategies

The theoretical landscape of quantum data encoding has been shaped by a sequence of foundational papers. Schuld and Killoran [30] established the connection between quantum feature maps and kernel methods, showing that quantum circuits define kernels on data through the inner product of encoded states in Hilbert space. This framing—encoding as an implicit kernel—is the basis for all kernel-based QML methods.

Havlíček et al. [15] operationalized this idea with a concrete circuit construction: the ZZ feature map, which applies Hadamard layers followed by diagonal Pauli rotations encoding both linear (x_i^2) and cross-product ($x_i \cdot x_j$) terms. This circuit, equivalent to what we term IQP (Instantaneous Quantum Polynomial) encoding, was demonstrated on IBM quantum hardware and showed experimental advantage over classical SVMs on a constructed classification task.

Thanasilp et al. [33] later proved that global quantum kernels concentrate exponentially in the number of qubits: off-diagonal fidelities vanish and K becomes the identity. Appel [3] showed that this collapse is often *premature* relative to the data: the recorded width E is an embedding dimension, while the correlation fractal

dimension D_2 estimates the intrinsic dimension of the support. Encoding all E columns—or the PCA-95% width—in an angle or IQP map can kill the kernel before any label is seen. On Breast Cancer ($E = 30$), a one-layer ZZ fidelity kernel is geometrically dead by four qubits, while $D_2 \approx 2.5$ yields a three-qubit budget that stays alive [3]. FD-ASE [10] then chooses a subset of *original* columns whose partial dimension recovers D_2 ; unlike PCA, it does not return linear mixtures. The present system operationalizes that budget: $q^* = \max(2, \lceil D_2 \rceil)$ caps circuit width *before* encoding recommendation and KTA ranking.

Pérez-Salinas et al. [25] introduced data re-uploading, departing from the one-shot encoding paradigm. In their framework, classical data is inserted into the circuit multiple times across layers, interleaved with trainable parameters. Data re-uploading has become the de facto encoding for variational quantum classifiers and quantum neural networks.

LaRose and Coyle [20] conducted the first systematic comparative study of encoding strategies across multiple datasets and noise conditions, establishing that no single encoding dominates and that the optimal choice is dataset-dependent. Their work motivates the need for an automated selection mechanism—a gap this system addresses.

The most comprehensive treatment is Sammartino’s 2026 survey [28], which reviews 66 papers from 2017 to 2026, classifies encodings into six families, and derives practical selection guidelines. Crucially, Sammartino formalizes the critical gate error threshold: for hardware with gate error rate $p \geq p^* \approx 10^{-3}$, deep encodings (amplitude, IQP, custom feature map) accumulate noise faster than their expressibility advantage compensates, making shallow encodings (angle, dense angle, data re-uploading) preferable. Dense-angle encoding—which packs two features per qubit via $R_y(x_{2i}) \cdot R_z(x_{2i+1})$ —is identified as the most underused encoding family in applied QML despite its favorable depth-qubit tradeoff. Dense-angle and standard angle encodings also introduce *relative phases* between qubits (via R_z or multi-qubit entangling maps); how much classification signal lives in phase versus amplitude remains under-explored empirically in QML benchmarks—a gap we note but do not resolve here.

Recent work emphasizes that the classical map applied *before* $U_\phi(x)$ is as consequential as the circuit itself [9, 11, 20]. Our implementation applies min-max scaling per feature to $[0, \pi]$ before rotation-based encodings and reports degeneracy warnings. In quantum circuits gate order is generally non-commutative; for encoding, the order in which features are mapped onto qubits can change the implicit kernel even for a fixed ansatz family [12]. Our implementation (`encoding_optimization.py`) runs a three-phase greedy search—permutation, backward selection, and per-feature weights—maximizing KTA when labels are present; it is enabled via `optimize_features=true` on `/v1/compare/csv` and the `compare_csv_embeddings` tool. Havlíček et al. [15] describe IQP/Hamiltonian encodings with a specific ZZ Hamiltonian; broader Heisenberg/Ising and Projected Quantum Feature Maps [6, 11, 32] differ in preprocessing and optional initial states $|s\rangle$ —discussed as related work rather than implemented encodings here.

¹API: <https://github.com/anapaulaappel/quantum-encoding-agents>; Agents: <https://github.com/anapaulaappel/openclaw-quantum-agents>. All code, policies, agent configurations, and the Kubeflow pipeline were developed and are maintained by the author.

2.2 Encoding as an Implicit Kernel

A result fundamental to the theoretical grounding of this system is Schuld’s 2021 theorem [29]: every supervised quantum model that encodes data into a quantum state $|\phi(x)\rangle$ and measures an observable is mathematically equivalent to a kernel support vector machine with kernel:

$$K(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2. \quad (1)$$

This equivalence has a decisive practical implication: “*the way that data is encoded into quantum states is the main ingredient that can potentially set quantum models apart from classical machine learning models*” [29]. Furthermore, kernel-based training—which is what QSVM performs directly—is provably at least as good as variational circuit training for the same encoding. The choice of encoding is therefore not a preprocessing decision; it is the core modeling decision, equivalent to choosing a kernel in classical SVM. Product-state encodings are typically classically simulable; entangled maps such as full-pair IQP and custom feature maps are heuristically harder to simulate—the comparison report includes a classical-simulability assessment per encoding. This system makes this choice explicit and measurable—the Kernel-Target Alignment (KTA) score computed by `/v1/kernel` and the multi-encoding KTA ranking in `/v1/compare/csv` are direct, label-aware evaluations of the implicit kernel defined by any encoding on the actual data.

2.3 Trainability and Barren Plateaus

The trainability of quantum circuits is a central concern for encoding choices used in variational algorithms. McClean et al. [22] demonstrated that randomly initialized quantum circuits exhibit barren plateaus: exponentially vanishing gradients that make training infeasible beyond moderate qubit counts. Cerezo et al. [8] surveyed the landscape of variational quantum algorithms, establishing data re-uploading as the preferred input encoding for QNNs and VQCs due to its per-layer data injection that maintains gradient flow.

2.4 Expressibility, Entanglement, and the Encoding Tradeoff

Sim, Johnson and Aspuru-Guzik [31] define *expressibility* as the deviation of a parameterized quantum circuit’s output distribution from the Haar measure. Their key result: shallow circuits have low expressibility and are confined to structured submanifolds of the state space, which may be insufficient to separate classes. This is the complementary risk to the barren plateau: too-shallow encodings underfit in Hilbert space, while too-deep circuits become untrainable due to vanishing gradients [22]. Sammartino [28] adds the third axis: above $p^* \approx 10^{-3}$, the depth budget is set by hardware decoherence, not by expressibility requirements.

Larocca et al. [19] further establish that over-parameterized quantum neural networks undergo a phase transition from a barren plateau regime to a trainable regime as the number of parameters exceeds a critical threshold, directly linking encoding circuit depth to parameter count and trainability.

2.5 Quantum Advantage and Scope of NISQ-Era QML

Huang et al. [17] proved exponential quantum advantage in a specific learning task—predicting properties of quantum physical systems from quantum measurements—demonstrated on 40-qubit superconducting hardware. However, this advantage is task-specific: it applies when the data itself is quantum in origin, not when encoding classical tabular data into quantum circuits. Huang, Kueng and Preskill [16] establish complementary information-theoretic bounds showing that classical ML can match quantum ML on average over input distributions, but quantum advantage is possible worst-case.

For classical datasets on NISQ hardware—the regime this system targets—quantum advantage over classical methods has not been proven in general. Bowles, Ahmed and Schuld [7] show, in a large-scale study across 12 QML models and 160 datasets, that classical baselines frequently match or outperform quantum classifiers at current qubit scales, underscoring that encoding evaluation is not optional but essential. This system is therefore designed as a NISQ-era engineering tool for informed encoding selection, not as a claim of general quantum supremacy.

2.6 NISQ vs. Fault-Tolerant Quantum Computing

The `hardware_profile` constraints in this system encode NISQ-era realities that will shift significantly in the fault-tolerant regime. On current NISQ devices—characterized by gate error rates of 10^{-3} to 10^{-2} , qubit counts of 10–400, and no error correction—circuit depth is the primary engineering constraint. In the fault-tolerant era, with logical qubits suppressing error to arbitrarily low levels, this constraint disappears: amplitude encoding’s exponential qubit compression becomes viable, and the encoding choice is driven purely by expressibility and task alignment. This design is forward-compatible: passing `gate_error_rate: 0.0` returns recommendations based on expressibility alone; passing `gate_error_rate: 5e-3` (IBM Eagle) returns hardware-adjusted recommendations.

2.7 AI Agents for Scientific Tooling

The use of LLM-based agents as interfaces for scientific computing tools is an emerging paradigm. In quantum computing, recent work spans LLM-assisted architecture design [21], variational circuit agents [18, 27], QGAN ansatz optimization [34], broader surveys of AI for quantum computing [1, 2], and generative circuit synthesis [13]—adjacent to, but distinct from, encoding selection. **This paper’s scope is encoding strategy selection and kernel evaluation**, not autonomous circuit generation.

To our knowledge, this is the first open-source system to deploy multi-personality LLM agents specifically for hardware-aware quantum encoding recommendation with empirical KTA ranking on user datasets.

The OpenClaw agent framework [24] provides the infrastructure for agent deployment with persistent memory (via `MEMORY.md`), user calibration (via `BOOTSTRAP.md` and `USER.md`), and skill encapsulation.

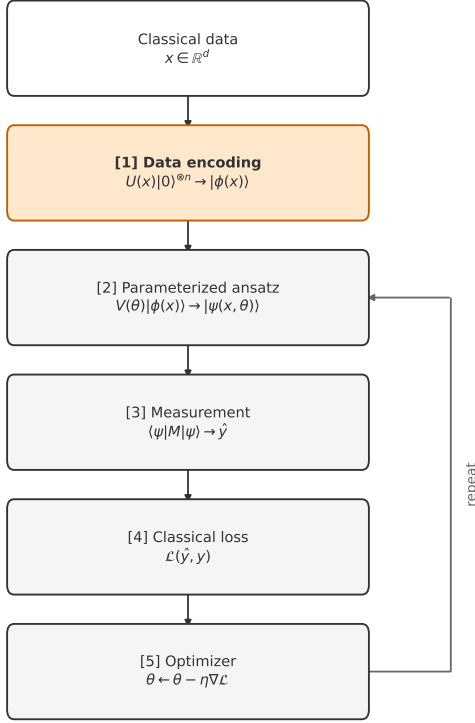


Figure 1: Supervised QML pipeline. Stage 1 (data encoding) is fixed during training and is the focus of this paper.

2.8 Secure Agent Deployment

NVIDIA OpenShell [23] addresses agent security through out-of-process policy enforcement: each agent runs in an isolated sandbox governed by a YAML policy that constrains filesystem access and network endpoints at HTTP method granularity, enforced by Landlock LSM and seccomp. The policy engine validates rules using the Z3 SMT solver before application. In this deployment, each agent receives a distinct policy: the expert agent (Circuit) is restricted to the Qiskit API and LLM endpoints only; the mentor agent (Quanta) additionally has read-only access to arXiv and Qiskit documentation.

3 Problem Description

3.1 The QML Training Pipeline

To position the encoding selection problem, I first establish the full QML training loop in which it occurs. A supervised quantum machine learning pipeline consists of five stages executed iteratively (Figure 1):

- (1) **Data Encoding:** $U(x)|0\rangle^{\otimes n} \rightarrow |\phi(x)\rangle$ (this paper)
- (2) **Parameterized Ansatz:** $V(\theta)|\phi(x)\rangle \rightarrow |\psi(x, \theta)\rangle$
- (3) **Measurement:** $\langle\psi(x, \theta)|M|\psi(x, \theta)\rangle \rightarrow \hat{y}$
- (4) **Classical Loss:** $\mathcal{L}(\hat{y}, y) \rightarrow \mathbb{R}$

- (5) **Classical Optimizer:** $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$ (then repeat)

The encoding layer (stage 1) is fixed for the duration of training—it is a hyperparameter of the model, not a trainable parameter. A poorly chosen encoding defines a kernel that cannot linearly separate the classes in Hilbert space, no matter how many training iterations the optimizer runs. This system operates at stage 1, with KTA providing a measurement of whether the chosen encoding creates a Hilbert space geometry useful for the loss at stage 4.

For quantum kernel methods (QSVM), stages 2–5 are replaced by classical SVM training on the kernel matrix $K_{ij} = |\langle\phi(x_i)|\phi(x_j)\rangle|^2$, computed by the `/v1/kernel` endpoint. In this case the encoding is the *entire* model specification, reinforcing the centrality of stage 1.

3.2 The Encoding Selection Problem

Let $\mathcal{D} = \{x_i\}_{i=1}^N \subset \mathbb{R}^d$ be a classical dataset with N samples and d features. A quantum encoding is a parameterized map $\phi: \mathbb{R}^d \rightarrow \mathcal{H}$ from the feature space to a Hilbert space $\mathcal{H}$ of dimension 2^n , implemented as a quantum circuit $U(x)$ acting on n qubits such that $|\phi(x)\rangle = U(x)|0\rangle^{\otimes n}$.

The encoding selection problem is: given $\mathcal{D}$ and an optional QML task specification $\mathcal{T}$ (classification, kernel method, variational circuit, etc.) and hardware constraints $\mathcal{Hw}$ (gate error rate, qubit count, connectivity topology), select an encoding ϕ^* that maximizes downstream task performance subject to hardware feasibility.

This problem is ill-posed for two reasons. First, the downstream task performance depends on the full learning pipeline, not just the encoding; it cannot be evaluated without training a model. Second, the interaction between data characteristics and encoding properties is non-linear and high-dimensional.

3.3 Current Practice and Its Limitations

In practice, encoding selection is rarely principled. A survey of QML implementations in the literature reveals that the majority use angle encoding as a default, regardless of dataset structure. However, this choice ignores:

- **Dimensionality:** for $d > 10$ features, angle encoding requires d qubits. Dense-angle encoding achieves the same depth with $\lceil d/2 \rceil$ qubits. More critically, d is the *embedding* width E : if the support has intrinsic dimension $D_2 \ll E$, an angle or IQP map of width E can collapse the fidelity kernel before labels enter [3, 33]. PCA-95% is not a substitute for D_2 .
- **Task alignment:** for QSVM, encodings without entanglement define kernels with limited expressibility. IQP and custom feature map encodings define richer kernels.
- **Hardware noise:** above $p^* \approx 10^{-3}$, amplitude encoding’s deep state preparation circuit accumulates noise faster than its qubit compression advantage is worth. On IBM Eagle hardware ($p \approx 5 \times 10^{-3}$), the effective circuit depth after error accumulation can reduce the signal-to-noise ratio below classification threshold. The fractal budget q^* and the chip limit `max_qubits` are independent caps.
- **Data type:** basis encoding, the natural choice for binary or categorical data, is systematically ignored when practitioners default to angle encoding for all data types.

3.4 The Accessibility Gap

Beyond the technical selection problem, there is an accessibility gap: the knowledge required to make principled encoding choices—distributed across a dozen papers published between 2018 and 2026, requiring familiarity with quantum information theory, circuit complexity, and NISQ hardware characteristics—is not accessible to the ML practitioner approaching QML for the first time.

This gap manifests in two distinct user profiles. The **QML practitioner** (expert) needs dense, metric-first information: encoding name, circuit depth, qubit count, KTA score, and a reference to the relevant paper. The **ML practitioner approaching QML** (learner) needs progressive disclosure: physical intuition before mathematics, the Bloch sphere before Dirac notation, a concrete analogy before a formal definition. Current tools address neither profile. The system I present bridges this gap through specialized agents calibrated to each profile.

4 Quantum Encoding Agents: System Design

4.1 Architecture Overview

The system consists of three loosely coupled layers (Figure 2): (1) an OpenClaw agent gateway with LLM backend (Qwen2.5-Coder-14B via Ollama, or Llama-3.2-3B via vLLM on OpenShift), (2) a FastAPI microservice (llamaqiskitagents) exposing the quantum pipeline, and (3) the NVIDIA OpenShell security layer enforcing per-agent sandbox policies. The microservice is stateless and can be called directly via HTTP independently of the agent framework. Nine quantum tools are registered under `/v1/tools` and `/v1/tools/dispatch`; multi-turn orchestration is available via `/v1/agent/chat` and the web UI at `/chat` (agent mode with personas or direct CSV comparison without LLM). OpenClaw and other agent frameworks call the same tool surface over HTTP.

4.2 Data Profile Analysis

The first processing step is structured data profiling. Given input in any of three forms—a numpy array, a CSV file, or a natural language description—the system extracts a `DataProfile` dataclass with the structural flags `n_samples`, `n_features` (embedding dimension E), `is_binary`, `is_categorical`, `is_continuous`, `has_negative`, and `description`, plus optional fractal fields (D_2 , E , q^* , FD-ASE column indices and names, `fractal_selection_applied`). For text descriptions, keyword detection in both Portuguese and English maps phrases to profile flags. Fractal fields stay empty unless a 2-D numeric matrix is present ($N \gtrsim 32$, $E > 2$, not binary).

4.3 The Seven Encoding Strategies

The system implements seven encoding families as Qiskit `QuantumCircuit` objects. Table 1 summarizes their properties.

Dense-angle encoding [28] achieves half the qubit count of standard angle encoding at depth 2 by packing two features per qubit:

$$U_{\text{DA}}(x) |0\rangle^{\otimes \lceil d/2 \rceil} = \bigotimes_{i=0}^{\lceil d/2 \rceil - 1} R_z(x_{2i+1}) R_y(x_{2i}) |0\rangle_i. \quad (2)$$

IQP encoding [15] is implemented without external dependencies through manual decomposition of $R_{zz}(\theta) = CX \cdot R_z(\theta) \cdot CX$:

$$U_{\text{IQP}}(x) = H^{\otimes d} \cdot \prod_i R_z(x_i^2) \cdot \prod_{i < j} R_{zz}(x_i x_j) \cdot H^{\otimes d}. \quad (3)$$

All feature pairs (i, j) are connected by default; `pairwise='adjacent'` restricts R_{zz} to neighboring qubits for linear topologies. Before any circuit, min-max scaling to $[0, \pi]$ and degeneracy warnings are applied (§4.4).

4.4 Mathematical Preprocessing Pipeline

Before building $U_\phi(x)$, the system applies encoding-specific classical transforms: L2 normalization for amplitude encoding; min-max scaling per feature to $[0, \pi]$ for rotation-based encodings using column bounds from the dataset; and warnings for near-duplicate angles and NISQ angle resolution. When CSV labels are present, `/v1/compare/csv` ranks all encodings by KTA.

4.5 Fractal Qubit Budget

Angle and IQP encodings spend one qubit per encoded coordinate. If that width exceeds the intrinsic dimension of the cloud, the fidelity kernel $K_{ij} = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$ concentrates on the diagonal [3, 33]. The system therefore computes, classically and without labels, the correlation fractal dimension D_2 (LiBOC box-count [10]) and the FD-ASE forward-greedy subset E_s of *original* columns whose partial dimension recovers D_2 . The qubit budget is

$$q^* = \max(2, \lceil D_2 \rceil). \quad (4)$$

If $|E_s| > q^*$, E_s is truncated to the first q^* selected indices: q^* is a hard cap on circuit width. The recommendation tree then uses the *effective* width $n_{\text{used}} = |E_s|$ (not E): angle and IQP are proposed only when $n_{\text{used}} \leq q^*$; otherwise the agent prefers dense-angle or a custom feature map. Generated Qiskit circuits encode only the selected columns.

This step is unsupervised and runs *before* KTA. The labelled greedy search of §4.6 sees the FD-ASE crop, not a fiction of KTA on all E columns. PCA is not used: it would mix axes [3]. FD-ASE is skipped for a single row, $E \leq 2$, binary data, or a text-only description. Setting `apply_fractal_budget=false` leaves D_2 , q^* , and `selected_columns` on the profile as an advisory, but recommends and encodes the full width. Chip `max_qubits` still applies.

4.6 Feature Encoding Optimization

Following [12], the classical map before $U_\phi(x)$ includes column order, subset selection, and per-feature weights. With `optimize_features=true` and labels, `encoding_optimization.py` runs a three-phase greedy search maximizing KTA on a fixed encoding *after* the unsupervised FD-ASE crop (§4.5): exhaustive or hill-climbing permutation ($d \leq 7$ vs. larger d), backward column elimination, and coordinate weight grid $\{0.5, 1.0, 2.0\}$. The best plan is applied before min-max scaling and the standard seven-encoding comparison.

4.7 Hardware-Aware Recommendation Policy

The recommendation pipeline has three stages: fractal crop (when a matrix is present), data-driven base selection, and hardware-constrained refinement.

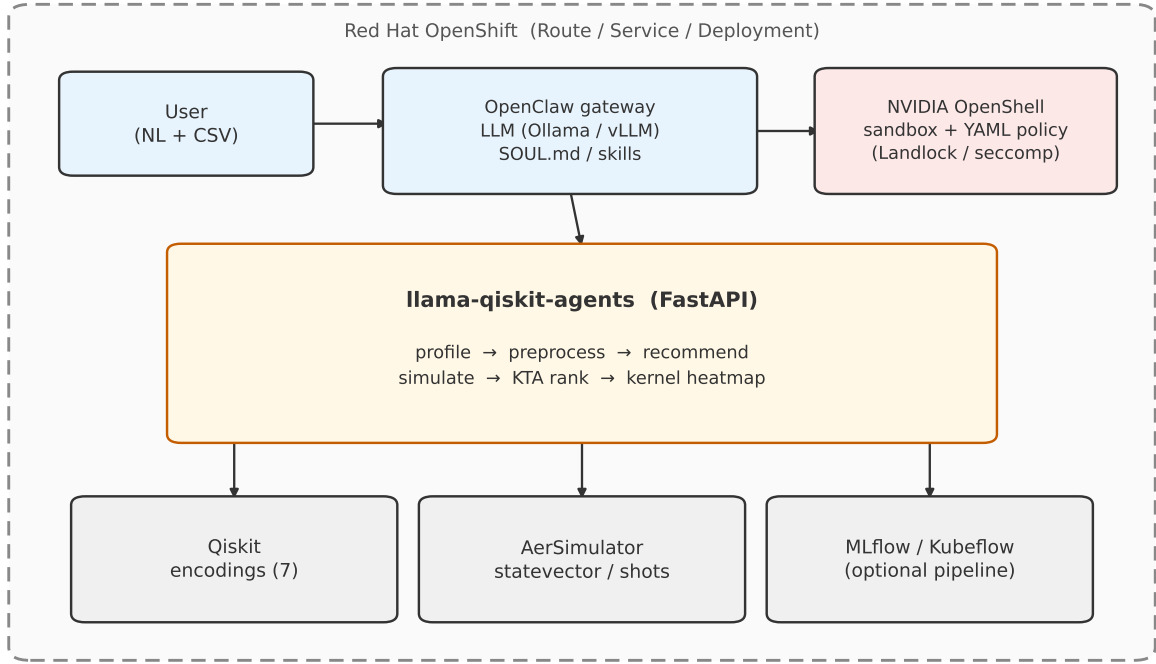


Figure 2: Deployment architecture. Agent gateways call the stateless encoding microservice (profile, D_2 /FD-ASE, recommend, KTA); OpenShell enforces per-agent sandbox policies on OpenShift.

Table 1: Quantum encoding strategies implemented in the system.

Encoding	Qubits	Depth	Recommended for	Key gates
Amplitude	$\lceil \log_2 n \rceil$	$O(2^n)$	Large vectors, qubit-constrained	StatePreparation
Angle	d	1	$d \leq 4$, continuous	$R_y(x_i)$ per qubit
Dense Angle [28]	$\lceil d/2 \rceil$	2	$5 \leq d \leq 12$, continuous	$R_y(x_{2i}) R_z(x_{2i+1})$
IQP [15]	d	$\sim 3d$	$8 \leq d \leq 16$, kernels	$H, R_z(x_i^2), R_{zz}(x_i x_j)$
Basis	d	$\leq d$	Binary / categorical	X per 1-bit
Data Re-upl. [25]	d	$\sim 4d$	VQC, QNN, variational	$R_y \times L + CX$ chain
Custom FM	d	$\sim 3d$	QSVM, arbitrary kernels	H, R_z, R_y, CZ pairwise

Base selection applies a priority decision tree over the DataProfile, using n_{used} after FD-ASE (otherwise E): (1) binary, $n_{\text{used}} \leq 16 \rightarrow \text{basis}$; (2) continuous, $n_{\text{used}} \leq 4$ and $n_{\text{used}} \leq q^* \rightarrow \text{angle}$; (3) continuous, $4 < n_{\text{used}} \leq 12$, no negatives, dense-angle fits $q^* \rightarrow \text{dense_angle}$; (4) single sample, $n_{\text{used}} \geq 4 \rightarrow \text{amplitude}$; (5) continuous, $8 < n_{\text{used}} \leq 16$ and angle-width $\leq q^* \rightarrow \text{iqp}$; (6) continuous, $n_{\text{used}} > q^*$ but dense-angle fits $\rightarrow \text{dense_angle}$; (7) continuous, $n_{\text{used}} > 16 \rightarrow \text{custom_feature_map}$; (8) default $\rightarrow \text{angle}$ only if it fits q^* .

Task refinement overrides based on the QML task: kernel methods and QSVM elevate to *custom_feature_map*; variational methods prefer *data_reuploading*; binary classification with binary inputs uses *basis*.

Hardware refinement applies the p^* threshold from [28]. For any HardwareProfile with $\text{gate_error_rate} \geq 10^{-3}$, deep encodings (amplitude, custom_feature_map, iqp) are replaced by *dense_angle* ($d > 4$) or *angle* ($d \leq 4$). Heavy-hex or linear connectivity triggers a SWAP overhead warning for custom_feature_map; max_depth_budget triggers a depth feasibility warning.

4.8 Natural Language Explanation Generation

The explanation layer generates a four-paragraph structured narrative in the detected language, each paragraph citing concrete values from the data profile and simulation result: (1) data description (including D_2 , q^* , and FD-ASE columns when available), (2) encoding justification with geometric intuition, (3) circuit metrics with

NISQ compatibility assessment, and (4) comparative analysis of alternatives with concrete depth and qubit differences. Language detection uses vocabulary overlap between the input text and Portuguese/English marker sets.

4.9 Quantum Kernel Evaluation

For datasets where classification labels are available, the system computes the quantum kernel matrix $K \in [0, 1]^{N \times N}$:

$$K_{ij} = |\langle \phi(x_i) | \phi(x_j) \rangle|^2. \quad (5)$$

Statevectors are computed using the StatevectorSimulator (exact, without measurement shots) and inner products are computed classically. When class labels are available, the system additionally reports the *Kernel-Target Alignment (KTA)*: a scalar, label-aware metric, bounded in $[-1, 1]$, that quantifies how closely the empirical kernel matrix K mirrors an idealized “target” kernel $T = yy^T$ built purely from class membership ($T[i, j] = +1$ if samples i, j share a class, -1 otherwise). Formally, it is the (Frobenius) cosine similarity between the two matrices:

$$\text{KTA}(K, y) = \frac{\langle K, yy^T \rangle_F}{\|K\|_F \cdot \|yy^T\|_F}, \quad (6)$$

where $y \in \{-1, +1\}^N$ is the label vector. Intuitively, $\text{KTA} \rightarrow 1$ indicates that the quantum feature map places same-class samples close together and different-class samples far apart in Hilbert space—evidence that the encoding is likely to yield a separable kernel for QSVM *before* any classifier is trained; $\text{KTA} \rightarrow 0$ indicates the kernel carries no information about the classes. The result is returned as a JSON matrix, a heatmap visualization (viridis colormap, per-class tick coloring), and a natural language caption.

In a preliminary evaluation on a synthetic two-class dataset ($N = 4, d = 2$), angle encoding achieved $\text{KTA} = 0.701$ while custom feature map achieved $\text{KTA} = 0.114$, demonstrating that richer encodings do not necessarily produce better-aligned kernels and that empirical evaluation via KTA is essential before committing to a training pipeline. Figure 3 shows the same contrast on a class-sorted Iris subset used only for display: block-diagonal structure tracks KTA, while the subset scores are not identical to the full- N values in Table 3. KTA is label-aware. Independently, `/v1/compare` reports the operational *kernel-alive* rule of [3]: the fidelity kernel is alive when the mean fidelity of Euclidean-nearest pairs is at least 0.25, the near/far ratio is at least 2, and the mean off-diagonal is at least 0.03. A high KTA on a dead kernel is a warning that class alignment is being measured on a nearly diagonal K . The same report includes a q -sweep of angle encoding under three views (FD-ASE original columns, PCA, and a CSV prefix) so q^* can be compared with the PCA-95% width that would otherwise over-allocate qubits.

4.10 Bloch Sphere and Q-sphere Visualization

For encodings with $n \leq 6$ qubits, the system optionally captures the statevector before measurement and renders the Bloch sphere of each qubit’s reduced state using `plot_bloch_multivector`. The resulting PNG (base64-encoded) accompanies the recommendation when `include_bloch: true` is set.

The Bloch picture is pedagogically useful but incomplete: each sphere is a *reduced* one-qubit state, so entanglement is invisible.

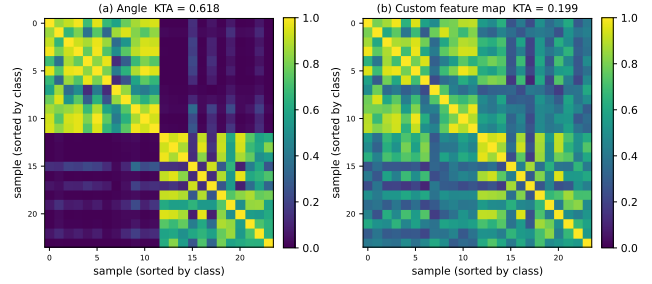


Figure 3: Fidelity kernels on a class-sorted Iris subset (12 samples per class; Aer statevector). (a) Angle encoding, subset KTA = 0.618. (b) Custom feature map, subset KTA = 0.199. Full- N KTA values appear in Table 3. Simulator only; not a QPU measurement.

The Q-sphere (`plot_state_qsphere`) plots the full 2^n amplitudes—latitude by Hamming weight, marker size by magnitude, colour by phase—so a product-state encoding and an IQP-style phase map look qualitatively different on the same sample (Figure 4). The API currently returns Bloch images; the Q-sphere panels are generated from the same Aer statevector used for kernel evaluation.

4.11 Agent Personalities and the Role of SOUL.md

Three OpenClaw agents expose the recommendation system through distinct communicative registers. **QiskitAgent** is the general-purpose agent. **Circuit** is calibrated for QML practitioners: its `SOUL.md` instructs it to presuppose familiarity with key concepts, respond in three dense blocks (encoding name, metrics, rationale), never explain superposition, and present trade-offs as two objective sides without choosing. **Quanta** is calibrated for learners: its `SOUL.md` prohibits the Schrödinger’s cat analogy (which lies) and mandates physical intuition before mathematics—the Bloch sphere before Dirac notation. Both agents perform a calibration ritual on first session (`BOOTSTRAP.md`) to populate `USER.md` with a persistent profile.

4.12 Skill Architecture

Two shared skills encapsulate reusable behavior across all agents. The `$qiskit-api` skill documents the recommendation endpoint as a `SKILL.md` file, eliminating documentation duplication and ensuring consistent behavior when the API evolves. The `$circuit-review` skill provides a four-step protocol for reviewing an existing circuit: identify the encoding, extract metrics, benchmark via `/v1/compare`, and issue one of three verdicts (keep / optimize / replace) with a technical rationale.

4.13 Security Architecture (OpenShell)

Each agent runs inside a dedicated NVIDIA OpenShell sandbox governed by a YAML policy validated by the Z3 SMT solver. The policies enforce least-privilege at HTTP method granularity: **Circuit** (expert) has access limited to `qiskit-api:8080` (read-write) and the LLM endpoint (POST only); **Quanta** (mentor) additionally

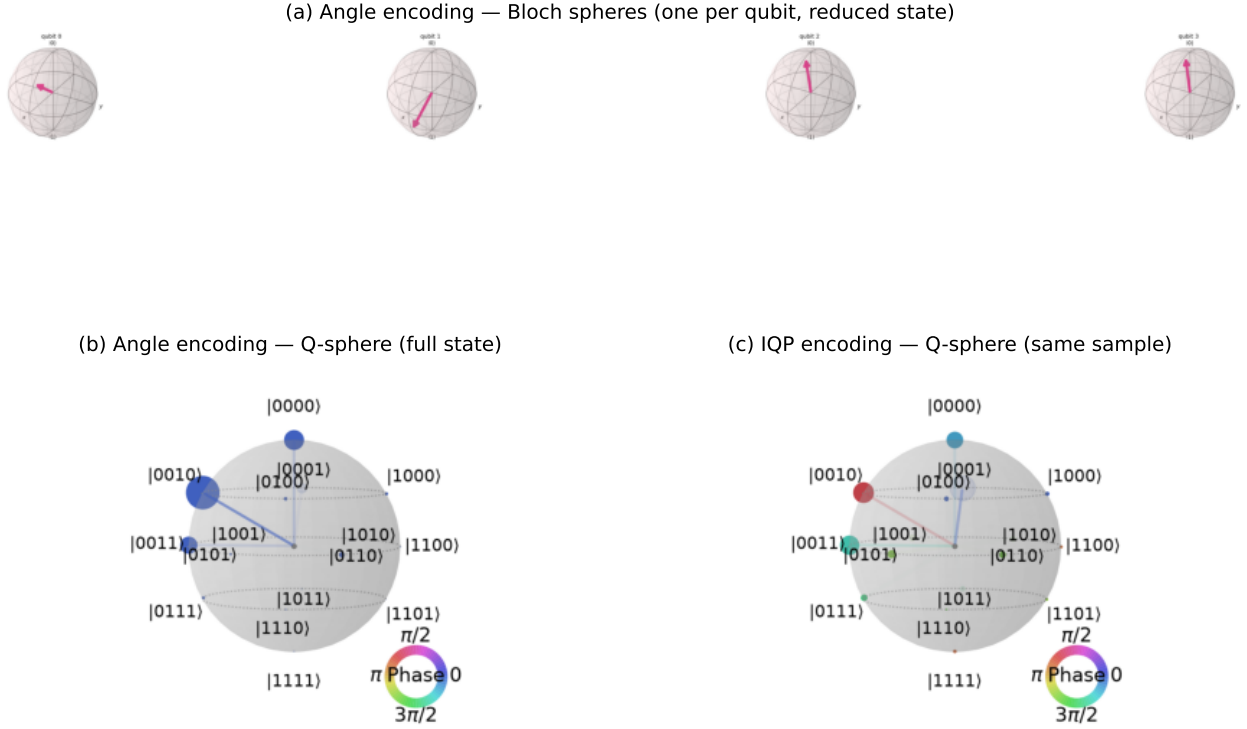


Figure 4: Same Iris sample encoded two ways (Aer statevector). (a) Angle encoding: four independent Bloch vectors (product state). (b) Q-sphere of that product state: amplitudes concentrated, uniform phase. (c) IQP on the same row: phases spread across basis states. Simulator only; not a QPU measurement.

has read-only access to arxiv.org and docs.quantum.ibm.com; QiskitAgent (general) includes write access to `/tmp` for CSV uploads. Binaries are specified as objects with path keys, enabling per-binary policy enforcement through Landlock LSM.

4.14 Kubeflow Pipeline for RHOAI

The full workflow is codified as a Kubeflow Pipelines v2 DAG deployable on Red Hat OpenShift AI [26]. Five sequential steps orchestrate the pipeline: (1) data profiling via `/v1/analyze`, (2) encoding recommendation via `/v1/recommend/explain`, (3) seven-encoding comparison via `/v1/compare`, (4) kernel matrix computation via `/v1/kernel` (when labels are provided), and (5) artifact logging to MLflow [36] (parameters, metrics, heatmap, Bloch sphere visualization, and generated Qiskit code). Hardware profile parameters propagate through all steps, enabling reproducible NISQ-aware experiments.

5 Discussion

5.1 The Role of KTA in Practice

My preliminary results suggest that KTA is a more reliable pre-training signal than circuit expressibility alone. Angle encoding (KTA = 0.701) outperformed the custom feature map (KTA = 0.114) on the test dataset despite the latter’s greater theoretical richness. This confirms LaRose and Coyle’s finding [20] that no encoding

universally dominates, and motivates the design approach: rather than recommending based on theoretical properties alone, I compute KTA on the actual data and present the score alongside the recommendation.

Benchmark suite (simulator). All KTA values in this section are computed with Qiskit’s Aer StatevectorSimulator via an exact fidelity kernel $K_{ij} = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$ unless a shot or noise estimator is named explicitly. They are *not* QPU counts unless a hardware table says otherwise. IBM Quantum validation uses the monthly budget in `benchmarks/HARDWARE.md` (encoding histograms plus a six-pair kernel-lite; not a full $N \times N$ Gram matrix). Public sklearn datasets (Iris binary, Breast Cancer top-6 variance, Wine binary, two moons) and a synthetic order-sensitive set are packaged under `benchmarks/` (python scripts/`run_benchmarks.py`). Shot, noise, and fake-backend transpile experiments are reproduced by `paper/figures/generate_nisq_experiments.py`. Table 2 reports the greedy order/selection/weight search (identity CSV order versus the optimized FeatureEncodingPlan). The *column-order* effect is isolated by the synthetic set, whose CSV places two noise columns *before* the two class-informative features.

On the synthetic CSV, two noise columns precede the signal (noise_x, noise_y, then feat_a, feat_b), so noise occupies the first qubits. The optimizer returns

Table 2: Feature-mapping optimization on the Aer simulator (scripts/run_benchmarks.py, at most 25 kernel samples; not QPU). KTA_{csv} uses the original column order; KTA_{opt} uses the searched plan on the same encoding.

Dataset	Encoding	N	d	KTA _{csv}	KTA _{opt}
Iris (binary)	angle	50	4	0.649	0.670
Synthetic (order)	dense angle	24	4	0.553	0.674
Breast Cancer (top-6)	IQP	40	6	0.535	0.542
Wine (binary)	IQP	40	13	0.204	0.550

feat_a $\rightarrow$ noise_y $\rightarrow$ feat_b $\rightarrow$ noise_x, placing an informative R_y at the start of each dense-angle qubit (Δ KTA = +0.121, about +22%). On Iris the sklearn order is already near-optimal: the plan keeps sepal width $\rightarrow$ petal length $\rightarrow$ petal width and *drops* sepal length (Δ KTA = +0.021). Breast Cancer keeps four of six variance-ranked features (area error $\rightarrow$ worst perimeter $\rightarrow$ mean perimeter $\rightarrow$ worst texture; Δ KTA = +0.006). Wine shows the largest *selection* gain: IQP on 13 chemical features collapses to color intensity $\rightarrow$ proline (0.204 $\rightarrow$ 0.550, Δ KTA = +0.346).

Agent choice versus alternatives. Table 3 compares the data-profile recommendation (/v1/recommend, the encoding the agent returns unless the user overrides the task) with three common alternatives, still on the simulator. On Iris and Breast Cancer the heuristic matches the highest KTA (angle 0.649; dense-angle 0.537). On the synthetic set the agent picks angle (0.547) while dense-angle is slightly ahead (0.553)—a near-tie that KTA ranking can surface. On Wine the heuristic selects IQP (0.204) because $d = 13$, but dense-angle wins KTA (0.399); the richer custom feature map (0.310) and IQP both trail the shallow packing. If the query names a kernel/QSVM task, refinement currently upgrades *all four* datasets to custom feature map—which is the *worst* of the four on Iris and synthetic, and only third on Wine. That gap is why the agent exposes empirical KTA rather than treating the heuristic as a final answer.

Fractal budget on the full Breast Cancer table. On the complete sklearn table ($E = 30$), estimate_fractal_budget yields $D_2 \approx 2.49$ and $q^* = 3$, matching the *a priori* budget in [3] (kernel alive near three qubits; dead at the PCA-95% width). FD-ASE keeps three original columns—worst concave points, worst texture, mean fractal dimension—and /v1/recommend returns **angle on 3 qubits**, not angle on 30. The default simulator suite (breast_cancer_fdase) now scores KTA and kernel-alive on that crop. Tables 2–3 retain the six-column variance subset used for the original feature-mapping comparison; they are a different protocol. With apply_fractal_budget=false the same D_2 and column list remain on the profile, but the circuit uses the full width.

QSVM versus a classical RBF baseline. KTA is a pre-training proxy; Table 4 reports five-fold stratified accuracy of an SVM on the precomputed Gram matrix versus sklearn’s RBF kernel (gamma=’scale’), still on Aer. Iris (setosa vs. versicolor) and the synthetic set are linearly easy: accuracy saturates near 1.0 for RBF and the shallow encodings, so KTA remains the ranking signal. Wine ($N = 40$) and Breast Cancer ($N = 40$) are harder for default RBF

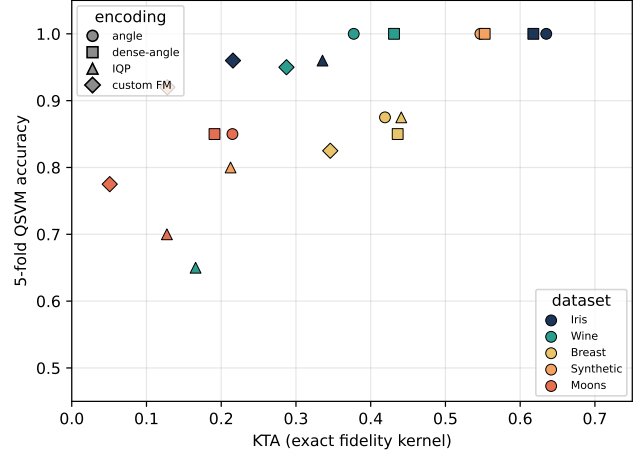


Figure 5: QSVM accuracy versus KTA for the 20 quantum kernels in Table 4 (Aer statevector). Spearman $\rho = 0.785$. Accuracy saturates on Iris and the synthetic set; moons sit at low KTA. Simulator only.

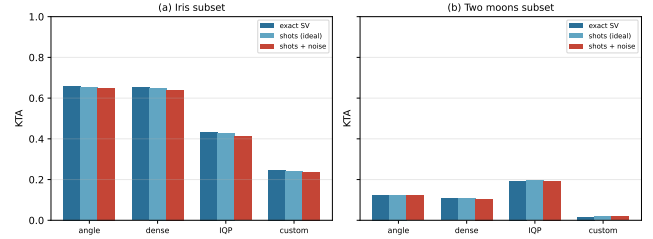


Figure 6: KTA ranking under exact statevectors, ideal shots, and noisy shots (same protocol as Table 5). Ranking is preserved; moons remain low-KTA for every encoding.

(0.650 and 0.675); angle, dense-angle, and (on Breast) IQP all beat RBF. Two moons ($N = 40$, $d = 2$, nonlinear) is the opposite check: RBF and the shallow encodings tie at 0.850, while IQP and the custom map trail—KTA is low for every quantum map (≤ 0.215). Across the 20 quantum kernels in Table 4, Spearman $\rho(\text{KTA}, \text{acc}) = 0.785$ (Figure 5). These are small- N simulator results, not hardware shots.

Shot-based kernels and Aer noise. The production endpoint uses an exact statevector fidelity. A hardware-like estimator is the compute-uncompute overlap $K_{ij} = \Pr(0^{\otimes n})$ after $U(x_j)^\dagger U(x_i)$, counted from shots. Table 5 and Figure 6 compare exact KTA with this estimator at 256 shots, then with a depolarizing plus readout noise model ($p_1 = 10^{-3}$, $p_2 = 10^{-2}$, $p_{r0} = 0.02$). On a stratified $N = 16$ slice the encoding ranking is unchanged: mean absolute error versus the exact Gram matrix is 0.012–0.022 (ideal shots) and 0.024–0.052 (noisy); noisy diagonals fall to about 0.92–0.98. KTA itself moves by at most 0.022. This is still Aer, not a QPU.

Transpile cost on a fake IBM-like backend. Table 6 reports logical depth and two-qubit count versus the ISA circuit after transpiling

Table 3: Agent encoding versus alternatives on the Aer statevector simulator (≤ 25 samples; no QPU). Bold: data-profile recommendation. Star: highest KTA among the four encodings shown.

Dataset	Angle	Dense-angle	IQP	Custom FM	Agent rec. (data)
Iris (binary)	0.649*	0.630	0.363	0.230	angle
Synthetic (order)	0.547	0.553*	0.213	0.128	angle
Breast Cancer (top-6)	0.526	0.537*	0.535	0.469	dense-angle
Wine (binary)	0.335	0.399*	0.204	0.310	IQP

Table 4: 5-fold stratified SVM accuracy on precomputed kernels (Aer statevector; sklearn SVC; RBF with default $\gamma = \text{'scale'}$). Not QPU.

Dataset	Metric	RBF	Angle	Dense-angle	IQP	Custom FM
Iris (binary, $N = 50$)	Acc.	1.000	1.000	1.000	0.960	0.960
Iris (binary, $N = 50$)	KTA	0.610	0.635	0.618	0.336	0.216
Wine (binary, $N = 40$)	Acc.	0.650	1.000	1.000	0.650	0.950
Wine (binary, $N = 40$)	KTA	0.163	0.377	0.431	0.166	0.287
Breast (top-6, $N = 40$)	Acc.	0.675	0.875	0.850	0.875	0.825
Breast (top-6, $N = 40$)	KTA	0.158	0.419	0.436	0.441	0.346
Synthetic (order, $N = 24$)	Acc.	1.000	1.000	1.000	0.800	0.920
Synthetic (order, $N = 24$)	KTA	0.120	0.547	0.552	0.212	0.128
Two moons ($N = 40$)	Acc.	0.850	0.850	0.850	0.700	0.775
Two moons ($N = 40$)	KTA	0.196	0.215	0.191	0.127	0.051

Table 5: Exact fidelity KTA versus compute-uncompute shot kernel (256 shots) and the same estimator under Aer depolarizing+readout noise. Stratified $N = 16$ subset. Not QPU.

Dataset	Encoding	Exact	Shots	Noisy shots
Iris	angle	0.659	0.655	0.651
Iris	dense-angle	0.653	0.649	0.640
Iris	IQP	0.434	0.430	0.412
Iris	custom FM	0.248	0.241	0.237
Moons	angle	0.123	0.124	0.122
Moons	dense-angle	0.106	0.106	0.104
Moons	IQP	0.194	0.196	0.194
Moons	custom FM	0.015	0.018	0.019

one sample to a 57-qubit heavy-hex GenericBackendV2 with IBM-like basis {id, rz, sx, x, cx} (optimization level 1; no live device). Angle and dense-angle stay depth 4 with zero two-qubit gates. IQP on Wine ($d = 13$) jumps from logical depth 72 / 156 two-qubit gates to ISA depth 190 / 377 CNOTs—the same encoding the data-profile heuristic selects because $d = 13$, and the encoding with the *worst* Wine KTA in Table 3. Dense-angle on seven qubits remains ISA depth 4. The hardware-aware agent should therefore prefer the shallow packing unless empirical KTA (and a depth budget) say otherwise.

IBM Quantum (ibm_fez). Table 7 reports the four week1_iris encoding circuits on IBM Quantum’s *ibm_fez* (open plan, 512 shots, 9 September 2026): one sample per class, CSV feature order versus the KTA-optimized plan (drop sepal length; simulator $\Delta\text{KTA} = +0.023$ on the 12-sample slice). All four circuits ran as a **single** Sampler job (dagota8mhr3c73e5fb20). Angle encoding transpiles

to ISA depth 5 with zero two-qubit gates. The dominant bitstring matches Aer on every circuit; the hardware peak probability is 5–11 points lower than Aer, consistent with readout noise rather than a changed mode. Table 8 is the kernel-lite that a full QPU Gram matrix cannot afford: six compute-uncompute overlaps $K_{ij} = \Pr(0^{\otimes n})$ on two samples per class, submitted as a single Sampler job (daa8m04e74ec73akhtd0, 30 August 2026). Intra-class fidelities stay ≈ 0.63 – 0.65 ; all four inter-class pairs remain ≈ 0 . The 4×4 mini-KTA is 0.691 (exact), 0.693 (Aer shots), and 0.688 on *ibm_fez*. IQP and Wine were not sent to the QPU: Table 6 already shows hundreds of CNOTs.

5.2 Limits of Text-Based Encoding Selection

The natural language input path—inferring data profile from a description—has inherent limitations. Keyword detection can misclassify ambiguous descriptions. D_2 and FD-ASE are not computed on text: without a 2-D array the fractal fields stay empty and the agent falls back to the previous heuristic. For production use, numerical data or CSV upload should be preferred; the text path is primarily useful for initial exploration and for the conversational interface.

5.3 Agent Personality as an Engineering Artifact

The SOUL .md-based personality specification is a form of behavioral engineering that sits between prompt engineering and agent architecture. Unlike a system prompt, SOUL .md is persistent (injected every session), versioned in git, and testable: a response from Circuit that begins with “Great question!” violates the soul specification and can be detected in evaluation. This framing— soul

Table 6: Logical encoding circuit versus ISA after transpiling to a 57-qubit heavy-hex GenericBackendV2 (IBM-like basis, optimization level 1). One representative sample. Not a live QPU.

Dataset	Encoding	n	Depth (log.)	2q (log.)	Depth (ISA)	2q (ISA)
Iris ($d = 4$)	angle	4	1	0	4	0
Iris ($d = 4$)	dense-angle	2	2	0	4	0
Iris ($d = 4$)	IQP	4	18	12	36	18
Iris ($d = 4$)	custom FM	4	8	6	32	12
Wine ($d = 13$)	angle	13	1	0	4	0
Wine ($d = 13$)	dense-angle	7	2	0	4	0
Wine ($d = 13$)	IQP	13	72	156	190	377
Wine ($d = 13$)	custom FM	13	26	78	241	300

Table 7: Angle-encoding histograms on IBM `ibm_fez` versus Aer (512 shots, 9 September 2026). Dominant bitstring agrees; hardware peak probability is slightly lower. One Sampler job: `dagota8mhr3c73e5fb20`.

Circuit	n	ISA depth	Mode	P_{QPU}	P_{Aer}
Class 0, CSV order	4	5	0010	0.670	0.783
Class 0, optimized	3	5	001	0.801	0.885
Class 1, CSV order	4	5	1101	0.775	0.822
Class 1, optimized	3	5	110	0.867	0.977

Table 8: Kernel-lite on the same Iris angle encoding: compute-uncompute K_{ij} for two samples per class. Mini-KTA on the filled 4×4 Gram (diagonal = 1). One Sampler job on `ibm_fez`, 512 shots, 30 August 2026 (`daa8m04e74ec73akhtd0`).

Pair	Exact	Aer shots	<code>ibm_fez</code>
Intra class 0	0.633	0.654	0.631
Intra class 1	0.681	0.709	0.645
Inter 0_a-1_a	0.000	0.000	0.000
Inter 0_a-1_b	0.004	0.002	0.004
Inter 0_b-1_a	0.001	0.002	0.000
Inter 0_b-1_b	0.002	0.002	0.008
Mini-KTA (4×4)	0.691	0.693	0.688

as contract—enables more systematic quality assurance of agent behavior than ad-hoc prompting.

5.4 OpenShell Policy Design as Security-by-Data-Profile

The assignment of different network permissions to different agents based on their communicative role is a novel security design pattern: **policy as a function of agent persona**. Quanta requires external internet access (arXiv, Qiskit docs) because its role is pedagogical and citation-heavy. Circuit does not, because experts are expected to know the literature. This reflects the insight that the minimum necessary privilege of an AI agent is determined not just by its technical function but by its communicative purpose.

6 Conclusion

I presented Quantum Encoding Agents, a system that addresses the encoding selection problem in Quantum Machine Learning through

a combination of hardware-aware recommendation logic, a fractal qubit budget [3], natural language generation, quantum kernel evaluation, and specialized AI agents calibrated to different user expertise levels. The system is grounded in the theoretical framework of Sammartino [28], Havlíček et al. [15], and the D_2 /FD-ASE result that angle-encoded kernels collapse when the map is wider than the intrinsic dimension of the data [3, 33]. It implements seven encoding families including the underused dense-angle encoding, crops original columns to q^* before KTA, reports kernel-alive beside KTA in `/v1/compare`, and provides KTA-based encoding quality assessment via a REST API.

The multi-agent architecture demonstrates that AI agents deployed for scientific tools can and should have distinct communicative personalities calibrated to their user populations—not as a cosmetic feature, but as a substantive design choice that determines what information to surface, in what order, and with what degree of assumed prior knowledge. The security architecture extends this idea: agent policy is a function of agent role, not just of agent function.

Future work includes: (i) larger on-device kernels and additional backends beyond the four encoding circuits and six-pair kernel-lite on `ibm_fez`; (ii) the Q-Tucker shallow state preparation [5]; (iii) the Quantum Spectral Model [35] and spectral methods [4]; (iv) Hamiltonian Heisenberg/Ising encodings and covariant kernels [14]; (v) optional initial states $|s\rangle$ before the feature map as in PQFM literature [6, 11]; (vi) extending KTA-guided automatic selection under hardware constraints; and (vii) joint optimization of feature plan and encoding family beyond the current FD-ASE crop and fixed-encoding greedy search [12].

Acknowledgments

The author thanks José Victor Soares Scursulim for a careful review of an earlier draft of this article. His comments substantially

improved the treatment of encoding preprocessing, related work, and mathematical notation.

References

- [1] 2025. LLM-based quantum computing research. IEEE Xplore document 11249998. <https://ieeexplore.ieee.org/document/11249998>
- [2] Yuri Alexeev, Marwa H. Farag, Taylor L. Patti, Mark E. Wolf, Natalia Ares, Alán Aspuru-Guzik, Simon C. Benjamin, et al. 2025. Artificial intelligence for quantum computing. *Nature Communications* (2025). doi:10.1038/s41467-025-65836-3
- [3] Ana Paula Appel. 2026. Fractal dimension predicts quantum kernel collapse in angle-encoded data. arXiv:2609.00475 [quant-ph] doi:10.48550/arXiv.2609.00475
- [4] Vasilis Belis, Joseph Bowles, Rishabh Gupta, Evan Peters, and Maria Schuld. 2026. Spectral methods: crucial for machine learning, natural for quantum computers? arXiv:2603.24654 [quant-ph] doi:10.48550/arXiv.2603.24654
- [5] Carsten Blank and Israel F. Araujo. 2026. Tucker iterative quantum state preparation. arXiv:2602.09909 [quant-ph] doi:10.48550/arXiv.2602.09909
- [6] Larry Bowden, Qi Chu, Bernard Cena, et al. 2026. Machine Failure Detection Based on Projected Quantum Models. arXiv:2601.15641 [quant-ph] doi:10.48550/arXiv.2601.15641
- [7] Joseph Bowles, Shah Nawaz Ahmed, and Maria Schuld. 2024. Better than classical? The subtle art of benchmarking quantum machine learning models. arXiv:2403.07059 [quant-ph] doi:10.48550/arXiv.2403.07059
- [8] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. 2021. Variational Quantum Algorithms. *Nature Reviews Physics* 3 (2021), 625–644. doi:10.1038/s42254-021-00348-9
- [9] Axel Ciceri, Austin Cottrell, Joshua Freeland, Daniel Fry, et al. 2025. Enhanced fill probability estimates in institutional algorithmic bond trading using statistical learning algorithms with quantum computers. arXiv:2509.17715 [quant-ph] doi:10.48550/arXiv.2509.17715
- [10] Elaine Parros Machado de Sousa, Caetano Traina Jr., Agma J. M. Traina, and Christos Faloutsos. 2002. How to Use the Fractal Dimension to Find Correlations between Attributes. In *First Workshop on Fractals and Self-similarity in Data Mining (in conjunction with ACM SIGKDD)*. ACM, Edmonton, Canada, 26–30.
- [11] Andras Ferenczi, Dagen Wang, Mariya Bessonova, et al. 2025. Credit Default Prediction with Projected Quantum Feature Models and Ensembles. arXiv:2510.01129 [quant-ph] doi:10.48550/arXiv.2510.01129
- [12] Tommaso Fioravanti, Brian Quanz, Gabriele Agliardi, Edgar Andres Ruiz Guzman, Ginés Carrascal, and Jae-Eun Park. 2025. Quantum Feature Encoding Optimization. arXiv:2512.02422 [quant-ph] doi:10.48550/arXiv.2512.02422
- [13] Florian Fürtter, Gorka Muñoz-Gil, and Hans J. Briegel. 2024. Quantum circuit synthesis with diffusion models. *Nature Machine Intelligence* (2024). doi:10.1038/s42256-024-00831-9
- [14] Jennifer R. Glick, Tanvi P. Gujarati, Antonio D. Corcoles, Youngseok Kim, Abhinav Kandala, Jay M. Gambetta, and Kristan Temme. 2021. Covariant quantum kernels for data with group structure. *Physical Review A* 104, 2 (2021), 022403. doi:10.1103/PhysRevA.104.022403
- [15] Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. 2019. Supervised learning with quantum-enhanced feature spaces. *Nature* 567, 7747 (2019), 209–212. doi:10.1038/s41586-019-0980-2
- [16] Hsin-Yuan Huang, Richard Kueng, and John Preskill. 2021. Information-theoretic bounds on quantum advantage in machine learning. *Physical Review Letters* 126, 19 (2021), 190505. doi:10.1103/PhysRevLett.126.190505
- [17] Hsin-Yuan Huang, Richard Kueng, Giacomo Torlai, Victor V. Albert, and John Preskill. 2022. Quantum advantage in learning from experiments. *Science* 376, 6598 (2022), 1182–1186. doi:10.1126/science.abn7293
- [18] Marco Knipfer, Alexander Roman, Konstantin T. Matchev, Katia Matcheva, and Sergei Gleyzer. 2026. AI Agents for Variational Quantum Circuit Design. arXiv:2602.19387 [quant-ph] doi:10.48550/arXiv.2602.19387
- [19] Martin Larocca, Nathan Ju, Diego García-Martín, Patrick J. Coles, and Marco Cerezo. 2023. Theory of overparametrization in quantum neural networks. *Nature Computational Science* 3, 6 (2023), 542–551. doi:10.1038/s43588-023-00467-6
- [20] Ryan LaRose and Brian Coyle. 2020. Robust data encodings for quantum classifiers. *Physical Review A* 102, 3 (2020), 032420. doi:10.1103/PhysRevA.102.032420
- [21] Zhiding Liang, Jinglei Cheng, Rui Yang, et al. 2023. Unleashing the Potential of LLMs for Quantum Computing: A Study in Quantum Architecture Design. arXiv:2307.08191 [quant-ph] doi:10.48550/arXiv.2307.08191
- [22] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. 2018. Barren plateaus in quantum neural network training landscapes. *Nature Communications* 9 (2018), 4812. doi:10.1038/s41467-018-07090-4
- [23] NVIDIA Corporation. 2026. NVIDIA NIM on Red Hat OpenShift AI. <https://developer.nvidia.com/nim>
- [24] OpenClaw. 2026. OpenClaw AI Assistant. <https://docs.openclaw.ai>
- [25] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. 2020. Data re-uploading for a universal quantum classifier. *Quantum* 4 (2020), 226. doi:10.22331/q-2020-02-06-226
- [26] Red Hat, Inc. 2026. Red Hat OpenShift AI. <https://www.redhat.com/en/products/ai/openshift-ai>
- [27] Kenya Sakka, Wataru Mizukami, and Kosuke Mitarai. 2026. An LLM System for Autonomous Variational Quantum Circuit Design. arXiv:2606.13380 [quant-ph] doi:10.48550/arXiv.2606.13380
- [28] Vincenzo Sammartino. 2026. Feature Encoding in Quantum Machine Learning: A Survey and Practical Guidelines. arXiv:2606.05387 [quant-ph] doi:10.48550/arXiv.2606.05387
- [29] Maria Schuld. 2021. Supervised quantum machine learning models are kernel methods. arXiv:2101.11020 [quant-ph] doi:10.48550/arXiv.2101.11020
- [30] Maria Schuld and Nathan Killoran. 2019. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters* 122, 4 (2019), 040504. doi:10.1103/PhysRevLett.122.040504
- [31] Sukin Sim, Peter D. Johnson, and Alán Aspuru-Guzik. 2019. Expressibility and Entangling Capability of Parameterized Quantum Circuits for Hybrid Quantum-Classical Algorithms. *Advanced Quantum Technologies* 2, 12 (2019), 1900070. doi:10.1002/qute.201900070
- [32] Anton Simen, Carlos Flores-Garrigos, et al. 2025. Quenched Quantum Feature Maps. arXiv:2508.20975 [quant-ph] doi:10.48550/arXiv.2508.20975
- [33] Supanut Thanasilp, Samson Wang, M. Cerezo, and Zoe Holmes. 2024. Exponential concentration in quantum kernel methods. *Nature Communications* 15 (2024), 5200. doi:10.1038/s41467-024-49287-w
- [34] Kento Ueda and Atsushi Matsuo. 2025. Optimizing Ansatz Design in Quantum Generative Adversarial Networks Using Large Language Models. arXiv:2503.12884 [quant-ph] doi:10.48550/arXiv.2503.12884
- [35] Peiyong Wang, Udaya Paramalli, and Casey R. Myers. 2026. Quantum Spectral Model: Data Reuploading with Input-Conditioned Frequency Support. arXiv:2607.22516 [quant-ph] doi:10.48550/arXiv.2607.22516
- [36] Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar. 2018. Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin* 41, 4 (2018), 39–45.